

AIM2- Preparations

Week 2, Day 4- EDA / Data Analysis

1. What is EDA?

EDA = Exploratory Data Analysis

Imagine somebody gives you a dataset containing 10,000 student records.

Before building a machine-learning model, you shouldn't immediately start training something.

First, you want to understand:

What does this dataset actually contain?

Typical questions are:

How many rows are there?
What columns exist?
What are the data types?
Are there missing values?
What are typical values?
What is the highest/lowest value?
How spread out are the values?
Which groups perform better?
Are there unusual values?
Do two variables seem related?

That investigation is EDA.

A common workflow is:

Raw Data

↓

Data Cleaning

← Day 3

↓	
EDA / Analysis	← TODAY
↓	
Visualization	← upcoming
↓	
Machine Learning	← Week 3

2. Today's Dataset

Create a new Python file or notebook and start with:

```
import pandas as pd

data = {
    "Name": ["Krishna", "Rahul", "Priya", "Amit", "Neha",
            "Ravi", "Sara", "John"],

    "City": ["London", "Toronto", "London", "Ottawa",
            "Toronto", "London", "Ottawa", "Toronto"],

    "Math": [85, 72, 95, 65, 88, 78, 92, 70],

    "Python": [92, 75, 98, 70, 90, 82, 95, 73],

    "AI": [88, 70, 96, 68, 91, 80, 94, 72]
}

df = pd.DataFrame(data)
```

We're intentionally using a **clean dataset** today. Day 3 handled cleaning; now our goal is analysis.

3. First Step — Understand the Dataset

You've already used these:

```
df.head()  
df.tail()  
df.shape  
df.columns  
df.dtypes  
df.info()  
df.describe()
```

We aren't relearning them. Instead, understand their role in EDA.

Think of it like troubleshooting:

```
Dataset arrives  
↓  
"What am I working with?"  
↓  
shape  
columns  
dtypes  
info()  
describe()
```

For example:

```
print(df.shape)
```

might give:

```
(8, 5)
```

Meaning:

```
8 rows  
5 columns
```

4. describe() — More Important Than It Looked Yesterday

Run:

```
print(df.describe())
```

You'll see something similar to:

	Math	Python	AI
count	...		
mean	...		
std	...		
min	...		
25%	...		
50%	...		
75%	...		
max	...		

You already know mean, min, and max.

Today I want you to understand the other values.

count

Number of non-missing observations.

```
count = 8
```

means eight valid values were analyzed.

mean

Average:

sum of values / number of values

min and max

Lowest and highest values.

50%

This is the **median**.

For:

10 20 30 40 50

median:

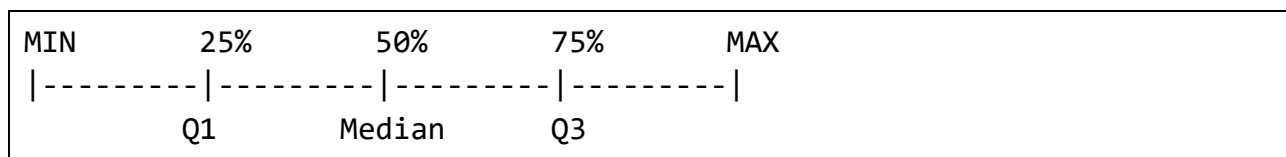
30

The median represents the middle of the ordered data.

25% and 75%

These are **quartiles**.

Conceptually:



They help us understand how values are distributed.

We will use these concepts much more when you reach ML.

5. Standard Deviation — std

This one is new and important.

Suppose two classes have:

Class A:

79, 80, 80, 80, 81

Class B:

40, 60, 80, 100, 120

Both could have approximately the same average.

But they're clearly very different.

Class A values are **close together**.

Class B values are **spread out**.

Standard deviation measures that spread.

Small std

→ values are relatively close to the mean

Large std

→ values are more spread out

You don't need to memorize the mathematical formula today.

For now understand:

Mean tells me the center; standard deviation tells me how spread out the values are around it.

6. Individual Statistical Analysis

We can analyze a single column:

```
print(df["Math"].mean())
print(df["Math"].median())
print(df["Math"].min())
print(df["Math"].max())
```

```
print(df["Math"].std())
```

You've seen most of these individually.

But EDA is about interpreting them together.

Suppose:

Mean = 80

Median = 82

Min = 30

Max = 98

Don't just print those numbers.

Ask:

Why is the minimum only 30 when most scores appear around 80?

Maybe it's valid.

Maybe it's an outlier.

Maybe it's incorrect data.

EDA generates questions that we investigate.

7. `value_counts()` — Categorical Analysis



Now something new.

We have:

```
df["City"]
```

How many students come from each city?

Use:

```
print(df["City"].value_counts())
```

You might get:

Toronto	3
London	3
Ottawa	2

This is extremely useful for categorical columns.

You can also calculate percentages:

```
print(df["City"].value_counts(normalize=True))
```

You might get approximately:

Toronto	0.375
London	0.375
Ottawa	0.250

Multiply by 100 conceptually:

Toronto	→ 37.5%
London	→ 37.5%
Ottawa	→ 25%

8. unique() vs nunique()

Another useful pair.

```
print(df["City"].unique())
```

returns the actual unique values:

```
['London' 'Toronto' 'Ottawa']
```

While:


```
print(df["City"].unique())
```

returns:

3

Remember:

unique()

→ **WHAT** unique values exist?

nunique()

→ **HOW MANY** unique values exist?

9. Creating an Average

You've already done this:

```
df["Average"] = df[["Math", "Python", "AI"]].mean(axis=1)
```

Now we're going to **analyze** that column.

```
print(df["Average"].mean())
print(df["Average"].median())
print(df["Average"].min())
print(df["Average"].max())
```

And:

```
print(df.sort_values("Average"))
```

or **highest first**:

```
print(df.sort_values("Average", ascending=False))
```

This introduces:

sort_values()

```
df.sort_values("Average")
```

Lowest → highest.

```
df.sort_values("Average", ascending=False)
```

Highest → lowest.

10. Finding the Highest/Lowest Student

You already know:

```
df["Average"].max()
```

But that only gives the number.

What if you want the student row?

One approach using your existing filtering knowledge:

```
highest = df["Average"].max()

print(df[df["Average"] == highest])
```

There's also:

```
df.loc[df["Average"].idxmax()]
```

and:

```
df.loc[df["Average"].idxmin()]
```

idxmax()

Returns the index containing the maximum value.

`idxmin()`

Returns the index containing the **minimum** value.

Conceptually:

`max()`

→ What is the **highest VALUE**?

`idxmax()`

→ Where is the *highest value*?

Then:

`df.loc[...]`

retrieves that row.

11. Grouping — `groupby()` ★ ★ ★

This is probably today's most important new Pandas concept.

Suppose the question is:

What is the average Math score for students from each city?

We need to:

Separate students by City

↓

Calculate Math mean
for each group

Pandas:

```
print(df.groupby("City")["Math"].mean())
```

Conceptually:

London

Krishna 85

Priya 95

Ravi 78

→ calculate mean

Toronto

Rahul 72

Neha 88

John 70

→ calculate mean

Ottawa

Amit 65

Sara 92

→ calculate mean

That's `groupby()`.

Think of it almost like:

GROUP → CALCULATE

12. Group Multiple Columns

What if you want Math, Python, and AI averages by city?

<pre>print(df.groupby("City")[["Math", "Python", "AI"]].mean())</pre>

Now Pandas groups students by City and calculates the mean of each selected numerical column.

You can also use:

```
df.groupby("City")[["Math", "Python", "AI"]].max()
```

or:

```
df.groupby("City")[["Math", "Python", "AI"]].min()
```

or:

```
df.groupby("City")[["Math", "Python", "AI"]].count()
```

This becomes extremely powerful with real datasets.

13. `agg()` — Multiple Calculations Together

Instead of separately doing:

```
df["Math"].mean()  
df["Math"].min()  
df["Math"].max()
```

you can use:

```
print(  
    df["Math"].agg(["mean", "min", "max"])  
)
```

And with groups:

```
print(  
    df.groupby("City")["Math"].agg(["mean", "min", "max"])  
)
```

Conceptually:

City	mean	min	max
London	...	...	...

Ottawa	...	...	...
Toronto	...	...	...

So:

`groupby()`

→ create groups

`agg()`

→ perform one or more calculations on those groups

14. Correlation — Introduction

One final EDA concept for today.

Suppose students who score highly in Python also seem to score highly in AI.

We might ask:

Is there a relationship between Python and AI scores?

Pandas can calculate:

```
print(
    df[["Math", "Python", "AI"]].corr()
)
```

Correlation generally ranges from:

-1 ----- 0 ----- +1

strong negative
relationship

little/no linear
relationship

strong positive
relationship

For example:

Python vs AI = 0.95

suggests a strong positive linear relationship in this dataset.

But this is extremely important:

Correlation does NOT prove causation.

If Python and AI scores correlate, we cannot automatically say:

Learning Python causes higher AI scores.

There could be many other explanations.

We'll revisit correlation later.

Day 4 Exercises

Now it's your turn. Write these yourself rather than copying solutions.

Use today's student dataset.

1. Print the dataset shape, columns and data types.
2. Run `describe()` and identify the **mean, median and standard deviation of Math**.
3. Find all unique cities and the number of unique cities.
4. Count students belonging to each city.
5. Calculate the percentage of students from each city using `value_counts(normalize=True)`.
6. Create an Average column from Math, Python and AI.
7. Sort students from highest Average to lowest.
8. Find the student with the highest Average using `idxmax()`.
9. Find the student with the lowest Average using `idxmin()`.
10. Calculate the average Math score for each City.
11. Calculate average Math, Python and AI scores for each City.
12. For each City, calculate the **mean, minimum and maximum Math score** using `agg()`.
13. Calculate the correlation matrix for Math, Python and AI.

Day 4 Understanding Questions

Answer these in comments underneath your code, like yesterday.

Q1. What is the difference between `mean()` and `median()`?

Q2. What does a high standard deviation tell us about the values?

Q3. What's the difference between:

```
df["City"].unique()
```

and:

```
df["City"].nunique()
```

Q4. Explain this in your own words:

```
df.groupby("City")["Math"].mean()
```

Q5. If Python and AI have a correlation of 0.95, does that prove that high Python scores **cause** high AI scores? Why or why not?

Day 4 Mini-Project — Student Performance EDA

Create your own EDA program using this same dataset.

Your program should answer:

STUDENT PERFORMANCE EDA

Dataset Overview

- Number of students
- Number of columns
- Unique cities

Overall Performance

- Overall Math average
- Overall Python average
- Overall AI average
- Overall student Average

Top/Bottom Performance

- Highest-performing student
- Lowest-performing student
- Top 3 students by Average

City Analysis

- Number of students per city
- Average Math/Python/AI scores per city
- Mean/min/max Math score per city

Relationships

- Correlation between Math, Python and AI

Don't worry about making graphs yet. **Visualization is coming later.**

Also, don't feel that you have to make the output look exactly like my outline. Structure it in a way that makes sense to you.